

Ball Machine

Madina har funnet opp en ballmaskin for å underholde IOI-deltakerne. Maskinens indre struktur er som følger:

- Maskinen inneholder N **noder**, indeksert fra 0 til $N - 1$. Node $N - 1$ er **rotnoden**.
- Hver u der $0 \leq u < N - 1$, har en **foreldrenode** $P[u]$ der $P[u] > u$. Noden u er et **barn** av $P[u]$. Rotnoden har ingen foreldre. En node uten barn kalles et **blad**.

Du kjenner **ikke** tily N eller foreldrenoden $P[u]$ til noen noder u . I stedet forteller Madina deg M , antallet blader. Indeksene til bladene er $0, 1, \dots, M - 1$. Din oppgave er å finne maskinens indre struktur ved å gjøre operasjoner på den.

Hver node i maskien kan holde maksimalt én **ball**, og hver ball har et ikkenegativt heltall som **verdi**. Man sier at en node har verdi x hvis den inneholder en ball med verdi x . En node er **opptatt** hvis den inneholder en ball, ellers er den **tom**. I begynnelsen er alle nodene tomme.

Du kan gjøre to operasjoner:

- insert**(U, X): prøv å sette inn en ball med verdi X på en bladnode U .
 - Hvis bladet U er opptatt vil operasjonen returnere `false` uten å sette inn en ball.
 - Hvis bladet U er fritt plasseres ballen i noden U . Deretter vil ballen flytte seg til foreldrenoden av noden den er i, helt frem til foreldrenoden er opptatt. Bevegelsen stopper når ballen når rotnoden eller når den når en node hvis forelder er opptatt. Denne operasjonen vil returnere `true`.
- collect**(): hvis rotnoden er tom (altså at maskinen er tom) vil `collect` returnere et tomt array. Ellers vil den rekursive funksjoen `traverse` slik beskrevet under kjøres. Denne samler verdiene av ballene som er i maskinen inn i et array S . Når du kjører operasjonen er S tomt og `traverse(N - 1)` blir kalt.

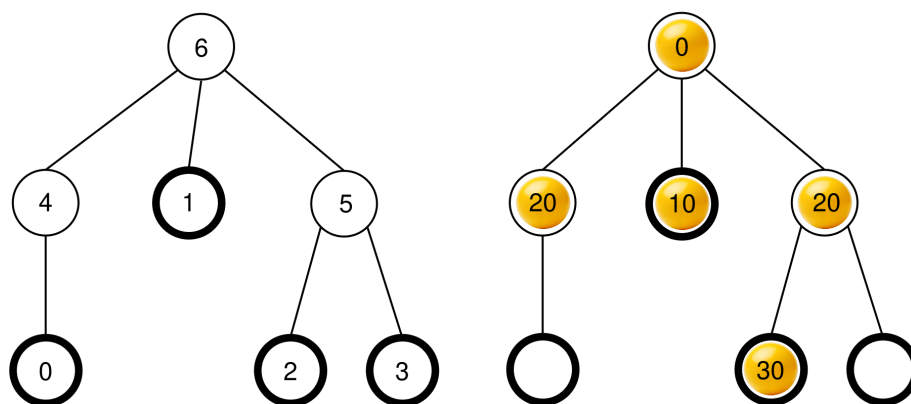
```

traverse(u):
    append the value of the ball at node u to the end of S
    let c[u] be the list of the occupied children of u
    sort c[u] in non-decreasing order according to the values
      of the balls at those nodes (if multiple nodes have the
      same value, they can appear in any order)
    for each v in c[u]:
        traverse(v)

```

Etter at `collect` har kjørt blir **alle ballene fjernet** fra maskinen og arrayet S returnert.

For eksempel, gitt en maskin med $N = 7$ noder og foreldre gitt ved $P = [4, 6, 5, 5, 6, 6]$. Bildet til venstre viser en tom maskin med indekserte noder, mens bildet til høyre viser en mulig ballplassering etter at noen `insert`-operasjoner har blitt kjørt. (Disse operasjonene er gitt i eksempel-seksjonen)



Når `collect()` blir kalt er rotnoden (node 6) opptatt, så S blir initialisert til $S = []$ og `traverse(6)` blir kalt.

traverse(6): node 6 har verdi 0; den legges til i S , noe som gir $S = [0]$. Barna til noden 6 er 4, 1, og 5. Alle disse er opptatte. Verdiene deres er henholdsvis 20, 10, og 20. Sortering i ikke-negativ rekkefølge gir $c[6] = [1, 5, 4]$ (merk at $c[6] = [1, 4, 5]$ også er gyldig, siden nodene 4 og 5 har samme verdi). Deretter kalles `traverse` på hver node i $c[6]$ i rekkefølgen gitt ved sorteringen.

- **traverse(1)**: node 1 har verdi 10; den legges til i S , noe som gir $S = [0, 10]$. Node 1 har ingen opptatte barn, så funksjonen returnerer.
- **traverse(5)**: node 5 har verdi 20; den legges til i S , noe som gir $S = [0, 10, 20]$. Node 5 har ett opptatt barn, node 2, så $c[5] = [2]$ og `traverse(2)` blir kalt.
 - **traverse(2)**: node 2 har verdi 30; den legges til i S , noe som gir $S = [0, 10, 20, 30]$. Node 2 har ingen opptatte barn, så funksjonen returnerer.
 - Kjøringen returnerer til `traverse(5)` som har prosessert alle barna i $c[5]$ og dermed returnerer.

- **traverse(4)**: node 4 har verdi 20; den legges til i S , noe som gir $S = [0, 10, 20, 30, 20]$. Node 4 har ingen opptatte barn, så funksjonen returnerer.

Alle kallene har returnert og det er ingen flere noder å prosessere. Til slutt blir alle ballene i maskinen fjernet og `collect` returnerer arrayet $S = [0, 10, 20, 30, 20]$.

Din oppgave er å finne N og gi et array av foreldre $R = [R[0], R[1], \dots, R[N-2]]$ som beskriver en mulig struktur i maskinen. Merk at benevningen (indekseringen) av nodene som hverken er blader eller rot kan være vilkårlig. Formelt er et array R korrekt hvis det er mulig å tildele distinkte benevninger $L[u]$ med $0 \leq L[u] < N$ for hver node u i maskinen slik at:

- $L[u] = u$ for alle $0 \leq u < M$ og $u = N-1$, og
- $L[P[u]] = R[L[u]]$ for alle $0 \leq u < N-1$.

Det er **ikke** et krav at $R[u] > u$. Maskien **må være tom** når du rapporterer svaret.

La K være antallet `collect`-operasjoner du gjør, og B den maksimale verdien til en ball over alle `insert`-kallene. La $C = K + B$. Da **må C ikke overstige 1000**. I noen deloppgaver avhenger scoringen av C .

Implementasjonsdeltaker

Du skal implementere følgende funksjon:

```
std::vector<int> find_structure(int M)
```

- M : antall blader i maskinen
- Funksjonen blir kalt eksakt én gang for hver testcase.
- Funksjonen må returnere en array $R = [R[0], R[1], \dots, R[N-2]]$ av lengde $N-1$, en korrekt array med foreldrenoder.

For å bruke maskien kan du bruke følgende to funksjoner.

Første funksjon er:

```
bool insert(int U, int X)
```

- U : indeksen til bladet der ballen skal settes inn. Her må $0 \leq U < M$.
- X : verdien til ballen. Her må $0 \leq X \leq 1000$.
- Funksjonen returner `true` hvis ballen ble satt inn i maskinen, eller `false` hvis bladnoden U allerede var opptatt.
- Denne funksjonen kan kalles maksimalt 500 000 times.

Den andre funksjonen er:

```
std::vector<int> collect()
```

- Samler verdien av alle ballene, tømmer maskinen og returnerer arrayet S .

Hvis verdien av C overstiger 1000 på noe tidspunkt i løpet av kjøringen vil løsningen din bli vurdert som `Output isn't correct: Too many resources used`.

Strukturen til maskien er bestemt før `find_structure` blir kalt. Graderen er altså **deterministisk** ved at den gitt samme sekvens av `insert`-operasjoner, alltid vil gi samme array som svar på `collect`.

Begrensninger

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Deloppgaver

Deloppgave	Poeng	Ytterligere begrensninger
1	5	Rotnoden har eksakt M barn.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Ingen andre begrensninger.

In deloppgave 3 og 4 avhenger poengene dine av C på følgende måte:

Deloppgave 3 (25 points)

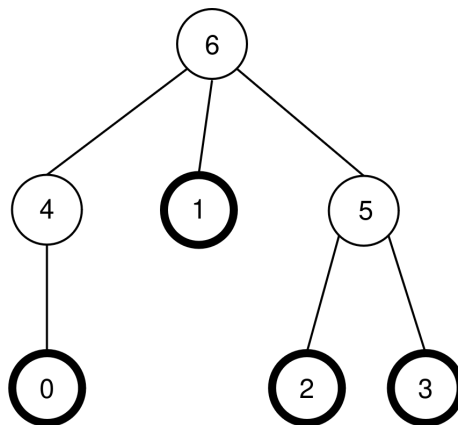
Begrensninger	Poeng
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Deloppgave 4 (60 points)

Begrensninger	Poeng
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Eksempel

Gitt en maskin slik som i oppgavebeskrivelsen over, altså $N = 7$, $M = 4$, og $P = [4, 6, 5, 5, 6, 6]$. Denne maskinen er gitt i figuren under:



Graderen gjør følgende kall:

```
find_structure(4)
```

Denne implementasjonen kan da gjøre følgende sekvens av kall:

1. **insert(0, 0)**: bladenoden 0 er ledig, så en ball med verdi 0 blir plassert i bladenode 0. Node $P[0] = 4$ er ledig, så ballen flytter seg til node 4. Node $P[4] = 6$ er ledig, så ballen flytter seg til node 6. Siden node 6 er rotnoden stopper ballens bevegelse. Funksjonskallet returnerer `true`.
2. **insert(3, 20)**: bladenoden 3 er ledig, så en ball med verdi 20 blir plassert i bladenode 3. Node $P[3] = 5$ er ledig, så ballen flytter seg til node 5. Siden $P[5] = 6$ er opptatt stopper ballens bevegelse. Funksjonskallet returnerer `true`.
3. **insert(1, 10)**: bladenoden 1 er ledig, så en ball med verdi 10 blir plassert i bladenode 1. Siden $P[1] = 6$ er opptatt blir ballen i node 1. Funksjonskallet returnerer `true`.

4. **insert(2, 30)**: bladenode 2 er ledig, så en ball med verdi 30 blir plassert i bladenode 2. Siden $P[2] = 5$ er opptatt blir ballen i node 2. Funksjonskallet returnerer `true`.
5. **insert(1, 25)**: bladenode 1 er opptatt, so the ball is not placed at leaf 1. Funksjonskallet returnerer `false`.
6. **insert(0, 20)**: bladenode 0 er ledig, så en ball med verdi 20 blir plassert i bladenode 0. Node $P[0] = 4$ er ledig, så ballen flytter seg til node 4. Siden $P[4] = 6$ er opptatt stopper ballens bevegelse. Funksjonskallet returnerer `true`.

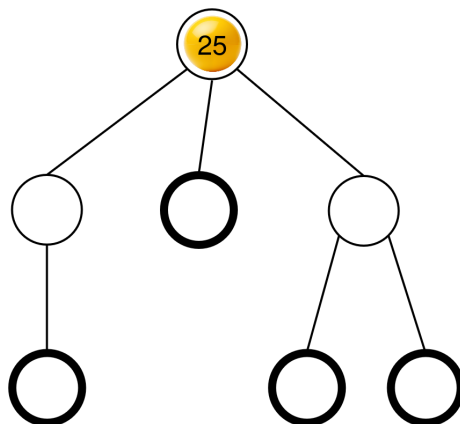
Den resulterende ballkonfigurasjonen har blitt vist i figuren tidligere i oppgavebeskrivelsen.

Videre kan implementasjonen gjøre følgende kall:

```
collect()
```

Kjøringen av denne funksjonen er beskrevet over, det returnerer $S = [0, 10, 20, 30, 20]$. Etterpå blir maskinen tømt for kuler.

Videre kan implementasjonen kjøre **insert(2, 25)**. Bladet 2 er ledig, så en ball med verdi 25 blir plassert i bladenode 2. Denne ballen beveger seg først til 5, og så til node 6 der den stopper. Funksjonskallet returnerer `true`. Den resulterende ballkonfigurasjonen er gitt i figuren under.



Til slutt kan implementasjonen kalle `collect()` som returnerer $S = [25]$ og tømmer maskinen.

Det er to korrekte foreldrenode-arrayer som `find_structure` kan returnere : $R = P = [4, 6, 5, 5, 6, 6]$ (som korresponderer til benevning $L = [0, 1, 2, 3, 4, 5, 6]$), og $R = [5, 6, 4, 4, 6, 6]$ (som korresponderer til benevning $L = [0, 1, 2, 3, 5, 4, 6]$). I dette eksempelet er $K = 2$ og $B = 30$, så $C = 32$.

Sample Grader

Inputformat:

```
N M  
P[0] P[1] P[2] ... P[N-2]
```

Outputformat:

```
K B C  
Q  
R[0] R[1] R[2] ... R[Q-1]
```

Her er Q lengden av arrayet R som `find_structure` returnerer.